Dense Distribution Labeling via DMS for Protein MLM

Xixian Jiang
Department of Biomedical Engineering
Duke University Pratt School of Engineering
xixian.jiang@duke.edu

Michael Kumoluyi
Department of Biomedical Engineering
Duke University Pratt School of Engineering
michael.kumoluyi@duke.edu

Ishaan Mehrotra
Department of Biomedical Engineering
Duke University Pratt School of Engineering
ishaan.mehrotra@duke.edu

Matias Vantilburg
Department of Computer Science
University of North Carolina at Chapel Hill
matias.vantilburg@duke.edu

Asher Wallen
Department of Biomedical Engineering
Duke University Pratt School of Engineering
asher.wallen@duke.edu

Sophia Vincoff
Department of Biomedical Engineering
Duke University Pratt School of Engineering
sophia.vincoff@duke.edu

Benjamin Perry
Department of Biomedical Engineering
Duke University Pratt School of Engineering
benjamin.perry@duke.edu

Zachary Quinn
Department of Biomedical Engineering
Duke University Pratt School of Engineering
zachary.quinn@duke.edu

Pranam Chatterjee
Department of Biomedical Engineering
Duke University Pratt School of Engineering
pranam.chatterjee@duke.edu

Abstract

For the final project, we developed a protein sequence generation pipeline that was driven by a fine-tuned version of EsmForMaskedLM. Our main objective was exploring the impact of adding densely labeled data on fine-tuning ESM. The data was constructed from the deep mutational scan (DMS), and allowed us to store information about other amino acids in a given position in addition to a traditional one hot marking. Using this finetuned model, our goal was to explore the effects DMS labeled fine-tuning on generative quality and sequence structure. Our generative model implemented an iterative unmasking strategy, progressively predicting masked residues using top-k sampling informed by model confidence. For every model we sweeped across various top-k values and found that increasing k resulted in a reduction in structure quality and an increase in sequence diversity. Despite this tradeoff, our results showed that fine-tuning with dense DMS labels enhanced the model's ability to create structurally valid, novel sequences during generation. Models trained with stronger dense supervision produced outputs with higher Shannon entropy, greater k-mer diversity, and improved pLDDT scores, indicating a more calibrated and expressive output distribution.

Preprint. Under review.

1 Introduction

Proteins are the fundamental functional units of life, and the ability to design or optimize them holds immense promise for therapeutic, industrial, and synthetic biology applications [11]. However, the fitness landscape of protein sequences is complex and highly context-dependent, making it difficult to predict how individual amino acid substitutions will affect stability, function, or binding. While large-scale language models trained on evolutionary sequence databases have shown impressive generalization in capturing structural and functional features, they are inherently limited by the assumption that wildtype residues are always the optimal choice [2]. This overlooks the growing body of high-throughput mutagenesis data showing that many non-wildtype variants can maintain—or even improve—function [9]. Incorporating this functional information into generative models represents a critical step toward more intelligent protein design [12].

Despite the success of protein language models in learning generalizable features from large-scale sequence databases, they are fundamentally constrained by their reliance on evolutionary consensus as a proxy for functionality [10]. These models are typically trained using masked language modeling objectives that treat the wildtype residue as the sole correct answer, ignoring the rich diversity of viable variants that exist outside of natural evolution. This approach fails to account for functional resilience and innovation captured in deep mutational scanning (DMS) datasets, which systematically explore the effects of single amino acid substitutions across entire proteins. While DMS provides direct, high-resolution measurements of mutational fitness, its integration into generative modeling has been limited by data sparsity and the challenge of aligning measured fitness values with probabilistic training objectives [8]. As a result, current models may overlook functionally beneficial mutations that deviate from wildtype, limiting their utility in truly optimizing proteins for novel tasks.

In this work, we propose a function-aware extension of the ESM2 masked language model by integrating residue-level fitness information from deep mutational scanning into the training objective. Rather than relying solely on one-hot wildtype labels, we fine-tune ESM2 using soft label vectors derived from DMS fitness scores, allowing the model to learn from both correct and "incorrect" amino acid substitutions. These soft targets, constructed by aggregating mutation-specific fitness data at each position, enable the model to recognize a spectrum of functionally viable residues—many of which diverge from the evolutionary consensus. By interleaving this DMS-based supervision with standard masked language modeling on large unlabeled proteins, and by replacing the cross-entropy loss with a mean squared error formulation where appropriate, our approach equips the model with a more nuanced understanding of local fitness landscapes. This modification is simple to implement, architecture-agnostic, and represents a step toward bridging the gap between generative modeling and empirical functional data.

2 Related Works

Recent advances in deep generative models for proteins have primarily focused on autoregressive transformers, variational autoencoders, and more recently, diffusion-based approaches. Autoregressive models such as ProtGPT2 and ProGen generate sequences residue by residue, capturing local context but lacking bidirectional understanding [3, 7]. Flow matching and diffusion models, including ProteinSGM and RFdiffusion, have demonstrated state-of-the-art performance in structure-guided sequence generation by modeling the denoising trajectory in either continuous or discrete sequence space [6, 13]. While these models are powerful, they often require extensive training pipelines and structure conditioning, limiting their accessibility and interpretability in sequence-only tasks. In contrast, masked language models like ESM2 offer a flexible and scalable alternative, encoding global sequence context via bidirectional attention and enabling powerful inpainting-style generation [5]. Our work builds on this masked modeling paradigm by introducing functional signal into the prediction objective itself—fine-tuning ESM2 with DMS-derived soft labels to align generative outputs with empirical fitness. To evaluate generation, we apply masked sampling followed by top-k residue selection at masked positions, ensuring that the model prioritizes substitutions that are both plausible and functionally informed. This lightweight, modular approach allows us to preserve the architectural strengths of ESM2 while augmenting it with experimentally grounded supervision, offering a distinct and accessible alternative to more complex generative pipelines that lack direct functional integration.

3 Methods

3.1 Data

3.1.1 Sparse Data

Our "sparse" - aka one-hot sequence - data came from UniRef50. UniRef50 is a sequence dataset built by clustering UniRef90 seed sequences that have at least 30% sequence identity to and 80% overlap with the longest sequence in the cluster. As UniRef90 was already a reduced version of UniRef100 based on clustering, we did not cluster, rather using the random split provided on Hugging Face. We filtered for sequences under length 1300, shuffled, and selected a train/validation/test split of 60000, 2000 and 500 respectively. The test split was also used pulled from during structure metric testing to form the base sequences for 'structural pseudoperplexity' tests.

3.1.2 Dense Data

Our dense labels were constructed from the deep mutational scan (DMS) data in the ProteinGYM benchmark Specifically, we selected only single-substitution DMS data, which provides a series of point-substitutions along a wild-type sequence and an associated fitness value per mutation. We claim this form of data is ideal for our problem of calibrating language models on protein stability. To motivate this, we formulate the prediction of fitness of mutants as follows:

Let $X = (X_1, X_2, \ldots, X_L)$ be a protein sequence of length L, and X_i the amino acid identity at position i. The remainder of the sequence, excluding position i, is denoted as $X_{\setminus i}$.

Under the single substitution setting:

$$\text{Fitness at position } i \propto P(X_i = a \mid X_{\setminus i}) \tag{1}$$

where a is the substituted amino acid.

This formulation naturally aligns with masked language modeling (MLM) or masked diffusion language modeling (MDLM) frameworks, where the model predicts the identity at position i while conditioning on the remainder of the sequence $X_{\setminus i}$. Fitness can be seen as a proportional "logit" to the probability $P(X_i = a \mid X_{\setminus i})$. By contrast, an autoregressive (AR) model predicts tokens sequentially, conditioning only on the positions it has generated so far. At the time of predicting position i, an AR model computes:

$$P(X_i = a \mid X_{<i}) \tag{2}$$

where $X_{<i}$ represents the subsequence of positions preceding i in the generation order.

This leads to a mismatch: while the experimental fitness is proportional to $P(X_i = a \mid X_{\setminus i})$, an AR model only approximates this via $P(X_i = a \mid X_{<i})$, which lacks access to downstream (future) positions $X_{>i}$. As a result, directly applying dense labels derived from DMS data in an AR framework would violate the true conditioning structure of the task.

In a similar vein, multiple simultaneous substitution data does not fit our goal. Extending our formulation to the positions i, j, k, we get:

$$\begin{aligned} P(X_i = a, X_j = b, X_k = c \mid X_{\setminus\{i,j,k\}}) = {} & P(X_i = a \mid X_{\setminus\{i,j,k\}}) \\ & \times P(X_j = b \mid X_i = a, X_{\setminus\{i,j,k\}}) \\ & \times P(X_k = c \mid X_i = a, X_j = b, X_{\setminus\{i,j,k\}}) \end{aligned} \tag{3}$$

Since the observed fitness value is only proportional to the joint probability of these substitutions, it is challenging to disentangle individual residue contributions. For instance, if $X_i = a$ is highly favorable but $X_j = b$ and $X_k = c$ are deleterious, learning their average effect as the true label might mislead the model and lead to worse representations. Indels can be phrased as a multiple-substitution of a slightly shorter sequence and thus have an identical issue.

Thus, we opt for masked language modeling on single substitution data only. Luckily, this is the majority of DMS data anyways - ProteinGYM, for instance, contains «<» and <»» multiple subs vs <» single subs. To string together these independent single substitution values, we consider the conditional probability distribution at each position i:

$$P(X_i \mid X_{\setminus i}) = \{P(X_i = a \mid X_{\setminus i}), P(X_i = b \mid X_{\setminus i}), \ldots, P(X_i = z \mid X_{\setminus i})\} \tag{4}$$

which forms a dense equivalent of a one-hot vector for position i, with one entry for the twenty standard residues. During training, we replace these probabilities with the experimentally observed fitness values, which are proportional to the underlying $P(X_i = a \mid X_{\setminus i})$ values. This results in a dense label vector $\mathbf{y}_i$ at each position i, where:

$$\mathbf{y}_i = \{\text{Fitness}(X_i = a), \text{Fitness}(X_i = b), \ldots, \text{Fitness}(X_i = z)\} \tag{5}$$

This process can be seen in depth in Fig. 1, with considerations for missing labels (which at training will simply be ignored) and mapping onto the ESM-2 vocabulary (which is just the residues plus special tokens which are also ignored).

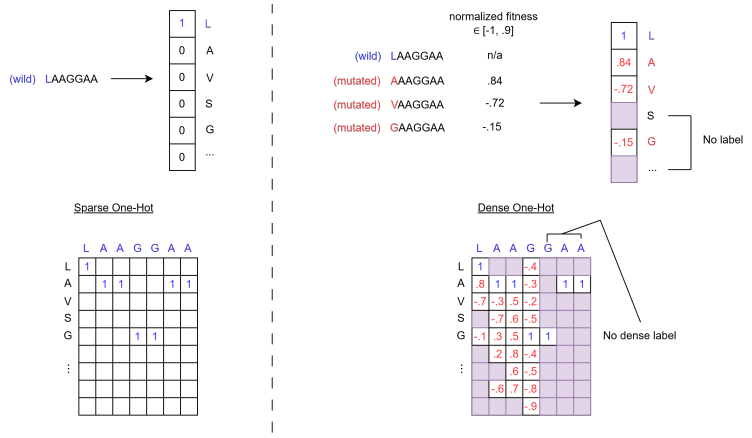


Figure 1: Standard sparse one-hot of a sequence VS our dense DMS label of the same sequence. Elements without labels—marked with purple—are ignored when calculating loss at train time.

To normalize across different fitness assays (which have different means & variances), we used a quantile transformation (to account for the distribution skew - mutant fitness values tend towards negative) followed by min-max scaling to the range [-1,.9]. Here, -1 represents the worst possible outcome for the model, while .9 represents the best achievable by a mutant. Leaving a gap between the best mutant and wild-type was a conservative design decision arising from true wild-type fitness being implicit and not measured in many of the assays. Since each assay included a large number of mutant fitness measurements, this scaling provided a good approximation of the underlying distribution within each assay.

In practice, we computed the wild types to densely label (which were not explicitly provided) via consensus sequence over a random twenty sampled sequences. Since only one position is mutated per sequence, these were essentially always identical. These densely labeled sequences were then clustered in a 90/10 train/validation split via MMseqs2. Clusters were run with 30% identity, less than 50% coverage, using connected component clustering and sensitivity 7.5. Lower identity (30% vs UniRef's 50%) was chosen as the coverage mode due to the relative diversity and small size of this set of proteins. Since the ultimate aim is to improve generative ability and not to correctly predict dense labels, no test set was necessary.

3.2 Thermostability Data

As presented in Fig.5, to benchmark how well our fine-tuned models capture protein properties, we implemented a single-layer Multilayer Perceptron (MLP) regressor using PyTorch to predict

the thermostability of proteins. To achieve this task, we utilized a curated meltome dataset with protein sequences and their melting temperatures, which is a protein property closely associated with thermostability that allows for regression tasks [1]. The dataset contains pre-split train, validation, and test sets. There are 53363 train sequences, 12794 validation sequences, and 3120 test sequences of various lengths and melting temperatures from the curated dataset. We did not filter by length.

3.3 Architecture

3.3.1 Finetuning

We fine-tuned the entirety of ESM-2 8M, unmodified. The model is a 6-layer encoder-only Transformer with 20 attention heads per layer, a hidden size of 320, and an intermediate size of 1280. It uses a vocabulary of 33 tokens, a maximum context length of 1026, and no dropout in hidden and attention layers. Only the loss function and input data were modified, refer to those sections.

Minimal model size was selected both due to compute constraints and to avoid an overparameterized model memorizing the relatively small dense dataset. We experimented with updating solely attention layers with LoRA, as well as freezing earlier layers, but found these worsened metrics, so were not pursued.

3.3.2 Classification Head

We used the fine-tuned models to embed protein sequences into 320-dimensional latent vectors. The model consisted of a single fully connected linear layer mapping the 320-dimensional input directly to a single scalar output, suitable for regression. No additional regularization techniques such as dropout or batch normalization were applied.

3.3.3 Sequence Generation

Sequence generation was done by implementing EsmForMaskedLM following MaskGIT [add source]. Initially a base (non-finetuned) esm model was passed into EsmForMaskedLM, then the weights were updated by our finetuned ESM model. To generate a sequence of length L, we initialized a sequence ,length L, of all masked tokens. Then we applied EsmForMaskedLM to produce a softmax across all tokens for every masked position. Following this we took the sum of the top k tokens and chose to unmask a percentage of positions with the highest sum. A distribution of the top k tokens was created and sampled to generate a new token for said masked position. This Process continued gradually unmasking more each time until the full sequence was unmasked. The first layer of the sequence generataion process is displayed in Fig Xxxxxxxxxxx

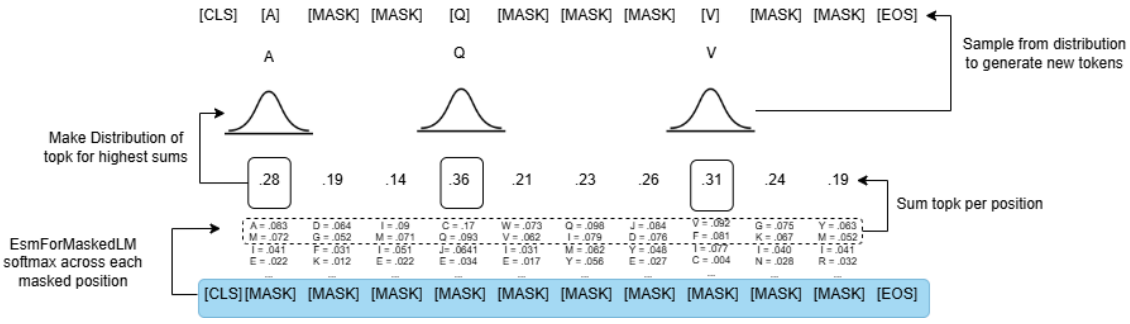


Figure 2: First layer of sequence generation done through customized EsmForMaskedLM implementation.

3.4 Training

We fine-tuned ESM-2 8M on UniRef50 sparse sequences and ProteinGYM-DMS dense sequences, with a batch size of 2. Optimization was performed with the AdamW optimizer, using a learning rate of 1e-5 and weight decay of 0.05. The model was trained for 10 epochs, with early stopping applied after 3 consecutive validation loss plateaus (threshold: 0.001 improvement). A custom cosine

ramp scheduler was used to anneal the contribution of an optional dense (DMS) MSE loss over training, which in testing improved performance by allowing model to adapt to easier MSE-over-CE formulation before moving onto the harder dense-prediction task.

At train time, we injected dense labels evenly throughout the epoch (so as to smooth updates compared to e.g. doing dense all at once), as per standard MLM loss. We only compared labels for dense with tokens that were both valid and mask tokens (e.g. not every indice is labelled in our dense dataset, see Fig. 1 for example) - see Fig 2. This was done rather than e.g. directly computing MSE on the dense and model prediction because indices which do not have an associated label contain no supervisory signal (e.g. the 0 value associated would be spurious - it would not actually indicate the indice is of neutral fitness).

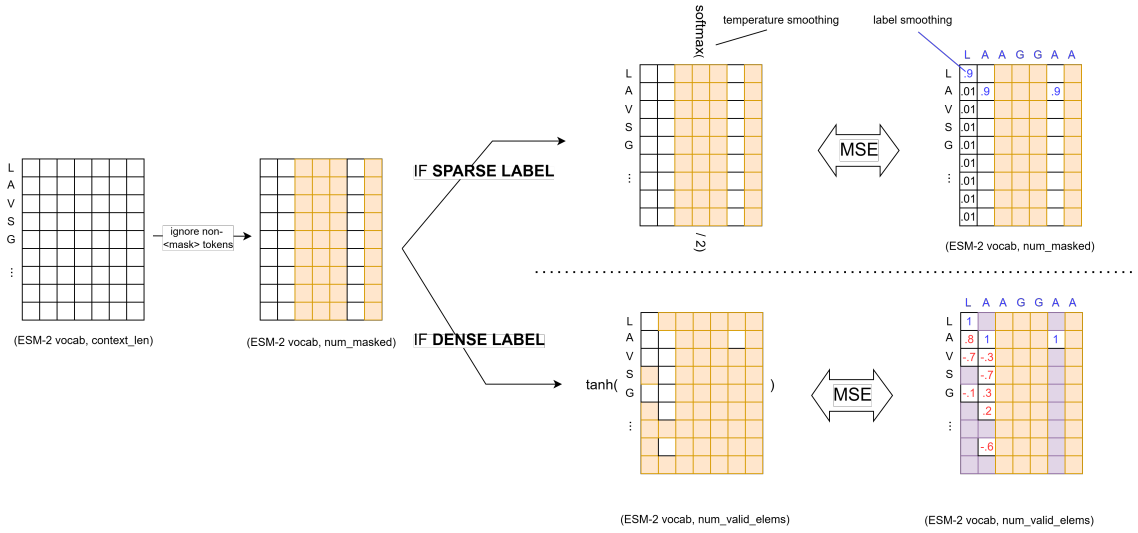


Figure 3: Standard sparse one-hot of a sequence VS our dense DMS label of the same sequence. Elements without labels - marked with purple - are ignored when calculating loss at train time.

We applied tanh to model predictions on the dense task in order to match the [-1,1] distribution of the prediction outputs (where -1 is a terrible choice and .9 is one of the highest fitness ones). We also used label and temperature smoothing on the sparse MSE formulation of next-token to make it closer to a cross-entropy-like loss and provide regularization. The aforementioned regularization & normalization efforts can be seen in Fig. 2.

3.5 Evaluation

3.5.1 Sequence Generation Metrics

To asses the quality of our generated sequences we used a combination of structure and diversity based metrics. Employing PLDDT, paE, and PTM for our structure based metrics, and shanon entropy, kmer diversity, and Jaccard similarity for our diversity metrics. They can bee seen computed in the bottom 3rd of Fig. 4., the model pipeline.

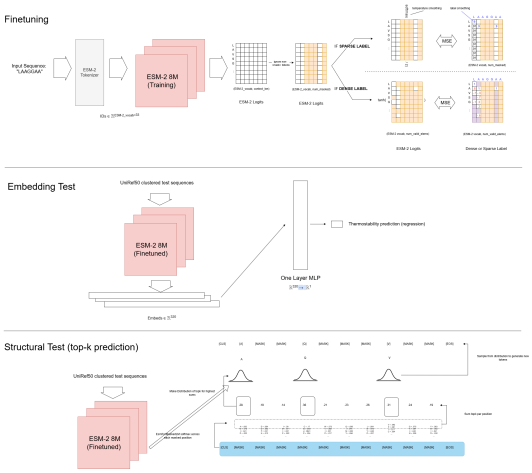


Figure 4: Model pipeline, data -> training -> evaluating.

In structure-based metrics, the predicted local distance difference test (pLDDT) measures confidence in the accuracy of each residue's position by comparing atom-pair distances in the predictions with those from the reference. For residue i, pLDDT is defined as:

$$\text{pLDDT}_i = 100 \left(1 - \frac{|d_{\text{pred}}(i) - d_{\text{ref}}(i)|}{d_{\text{ref}}(i)}\right)$$

where $d_{\text{pred}}(i)$ and $d_{\text{ref}}(i)$ are the distances between atom pairs in the predicted and reference structures, respectively. The mean pLDDT score is taken across all residues in a sequence.

The TM score is a measure of the global structural alignment between two structures, specifically how similar the overall fold of a predicted structure is to a reference structure. This score is less sensitive to local inaccuracies (like small misplacements of side chains) and focuses more on the overall fold. The TM-score is defined as:

$$\text{TM-score} = \max \left[\frac{1}{L_{\text{ref}}} \sum_{i=1}^{L_{\text{align}}} \frac{1}{1 + \left(\frac{d_i}{d_0}\right)^2} \right]$$

where d_i is the distance between the i-th pair of residues in the aligned structures, L_{align} is the number of aligned residues, L_{ref} is the length of the native reference structure, and $d_0(L_{\text{ref}})$ is a scale normalization factor typically defined as: $d_0(L_{\text{ref}}) = 1.24(L_{\text{ref}} - 15)^{1/3} - 1.8$ but often simplified to a constant such as 5Å [**?**].

However, because a true reference structure was not available for our model, we used a predicted TM-score (pTM), which was generated by a neural network head trained to estimate the TM-score using internal model features such as the pAE matrix. This learned pTM-score represents the global structural accuracy of the predicted fold, providing a structural confidence estimate even in the absence of an experimental baseline.

The predicted aligned error (pAE) estimates the expected error in residue-to-residue distances, calculated as:

$$\text{pAE} = \frac{1}{N} \sum_{i=1}^{N} |d_{\text{pred}}(i) - d_{\text{ref}}(i)|$$

where N is the number of residues in the structure. Lower pAE values indicate better predicted accuracy.

Together, pLDDT, pTM, and pAE offer complementary views: pLDDT quantifies per-residue confidence, pTM evaluates overall fold reliability, and pAE reflects the predicted residue-to-residue positioning error.

For sequence diversity, we employed three complementary metrics: Shannon entropy, kmer diversity, and Jaccard similarity. Shannon entropy at residue position i is defined as:

$$H_i = -\sum_{a \in A} p(a_i) \log p(a_i)$$

where A is the set of possible amino acids, and $p(a_i)$ is the probability of amino acid a at position i. Higher entropy indicates greater variability. Kmer diversity measures how many unique subsequences of length k appear in a sequence of length L.

$$\text{K-mer diversity}(k, L) = \frac{\text{unique K-mers}}{L - k + 1}$$

Jaccard similarity between sequences S_1 and S_2 based on k-mers of length k is:

$$\text{JaccardSim}(S_1, S_2) = \frac{|\text{K-mers}(S_1) \cap \text{K-mers}(S_2)|}{|\text{K-mers}(S_1) \cup \text{K-mers}(S_2)|}$$

where K-mers are subsequences of length k, and a higher Jaccard similarity means more shared local motifs, implying less diversity (with the caveat that in proteins motifs may be reasonable or even desired).

Collectively, these metrics capture sequence variability: entropy assesses diversity per position, K-mer diversity flags repetitive generation, and Jaccard similarity measures local motif overlap. Evaluating all three provides a comprehensive view of sequence diversity across generated proteins.

Additionally, we computed structure metrics again while applying a pseudo-perplexity-esque approach. Starting with sequences from the uniref database, we randomly masked tokens, then emplyed EsmForMaskedLM to predict said masked token, finally computing structure metrics.

3.5.2 Regression Head Metrics

The model architecture is strictly designed to be lightweight and efficient for the benchmarking of embeddings' representativeness of protein properties. It comprises one hidden projection layer and an output node. The input to the MLP is a 320-dimensional vector $x \in \mathbb{R}^{320}$ obtained by mean pooling the last hidden layer of our fine-tuned models. The regressor structure consists of:

- A fully connected layer projecting x to a 128-dimensional hidden vector:

$$h = W_1 x + b_1, \quad W_1 \in \mathbb{R}^{128 \times 320}, \quad b_1 \in \mathbb{R}^{128}$$

- A dropout layer applied to h with dropout probability $p = 0.3$:

$$h' = \text{Dropout}(h)$$

- A final linear transformation to a scalar output:

$$\hat{y} = W_2 h' + b_2, \quad W_2 \in \mathbb{R}^{1 \times 128}, \quad b_2 \in \mathbb{R}$$

Importantly, the model does not employ any non-linear activation functions between layers, resulting in a purely linear transformation chain regularized by dropout.

Training. The model was trained using a weighted Mean Squared Error (MSE) loss:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} w_i (\hat{y}_i - y_i)^2$$

where w_i is a sample-specific weight derived from the normalized absolute deviation of the target value from the training set median. This weighting strategy was implemented due to the over-representation of proteins with melting temperatures within the range of 40-60 °C. Optimization was performed using the Adam optimizer [4]. The learning rate and weight decay rate were kept consistently at 1e-3 and 1e-5 for embedding generated by different fine-tuned models. The maximum training epoch was set at 50 because we have observed training with most of the embeddings achieved minimal validation losses around epoch 30. Early stopping was implemented using validation losses with a patience of 20, which allowed training with some of the worse performing embeddings to escape local minima.

Evaluation. The model performances were evaluated on the pre-selected test sequences using the standard regression task metrics:

- **Mean Squared Error (MSE):** $\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- **Mean Absolute Error (MAE):** $\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$
- **Coefficient of Determination (R^2):** $R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$

R^2 directly indicates how well the embeddings capture the properties of the proteins related to their thermostability and melting temperatures, while MSE and MAE offer additional metrics for comparing the performance of different embedddings under the regression task.

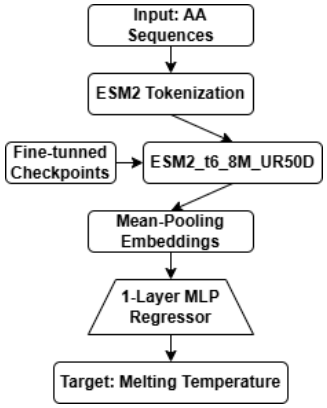


Figure 5: Simple 1-layer MLP workflow for protein melting temperature prediction and fine-tuned model embedding evaluation.

4 Results

Run	Psuedoperplexity	Test MSE	Test MAE	Test R^2
WD=0.05	6.95	97.29	7.79	0.2753
WD=0.01	7.02	109.06	8.38	0.1877
WD=0.005	6.90	106.10	8.13	0.2097
WD=0.0	6.84	103.86	7.99	0.2265
Dense=1	7.13	103.06	7.94	0.2324
Dense=0.5	6.95	97.29	7.79	0.2753
Dense=0.25	7.28	91.13	7.45	0.3213
Dense=0.1	6.77	94.04	7.65	0.2996
Dense=0.01	6.54	82.95	7.11	0.3821
Control (ESM-2)	6.37	79.12	6.94	0.41

Table 1: Psuedoperplexity (on next-token prediction, UniRef50) and one layer MLP results on thermostability prediction. Varying: weight decay and dense label importance weight in the loss. Control is ESM-2 fine-tuned via our method (e.g. ESM-3 masking rate, MSE), but without dense labels.

Run	Psuedoperplexity	pLDDT	PAE	pTM
WD=0.05	6.95	61.73	14.84	0.3716
WD=0.01	7.02	61.71	14.78	0.3724
WD=0.005	6.90	61.72	14.84	0.3713
WD=0.0	6.84	61.39	14.82	0.3704
Dense=1	7.13	61.53	14.85	0.3707
Dense=0.5	6.95	61.73	14.84	0.3716
Dense=0.25	7.28	61.51	14.91	0.3689
Dense=0.1	6.77	61.76	14.88	0.3702
Dense=0.01	6.54	61.76	14.83	0.3712
Control (ESM-2)	6.37	61.73	14.86	0.3706

Table 2: Next-token psuedoperplexity (from Table 1) and structural psuedo-perplexity metrics. Examined for models fine-tuned with varying importance of dense labels and varying levels of weight decay. Control is ESM-2 fine-tuned via our method but without dense labels.

top_k	pLDDT	PAE	pTM	Shannon Entropy	k-mer Diversity	Jaccard Similarity
1	93.65	5.63	0.592	0.0208	0.0233	1.000
3	48.10	18.62	0.245	0.3496	0.2770	0.404
5	44.77	19.51	0.217	0.4977	0.5815	0.223
6	47.64	18.66	0.235	0.5533	0.6835	0.144
12	45.19	20.38	0.203	0.7432	0.9132	0.043

Table 3: Control (ESM-2, fine-tuned without dense labels) performance on top k generative sampling as k increases.

top_k	pLDDT	PAE	pTM	Shannon Entropy	k-mer Diversity	Jaccard Similarity
1	94.11	4.92	0.639	0.0696	0.0575	1.000
3	78.72	11.16	0.444	0.4555	0.4502	0.401
5	56.80	15.09	0.322	0.6488	0.8423	0.121
6	54.37	15.31	0.308	0.6940	0.8952	0.081
12	45.05	16.77	0.268	0.8461	0.9804	0.018

Table 4: The Decay=.05 model (selected at random) performance on top k generative sampling as k increases.

top_k	pLDDT	PAE	pTM	Shannon Entropy	k-mer Diversity	Jaccard Similarity
1	93.42	6.16	0.578	0.0200	0.0222	1.000
3	65.13	13.61	0.379	0.5208	0.5554	0.265
5	57.12	15.42	0.314	0.6413	0.7949	0.182
6	54.33	15.47	0.306	0.6911	0.8711	0.113
12	47.92	16.55	0.276	0.8413	0.9782	0.020

Table 5: The Dense=.05 model (selected at random) performance on top k generative sampling as k increases.

Through the varying dense-weighting ablations in Tables 1 and 2 we see that, as dense labels play a larger role in the loss, the model becomes weaker in most respects. It loses representative capability, becoming gradually worse at predicting thermostability of proteins from embeddings (see MSE/MAE/R squared on Table 1). Through the lack clear upward trend as weight decay - a regularizing term - increases, we can see that this, surprisingly, does not appear to be a symptom of overfit on the small dense label dataset.

We might ask whether the model, in spite of losing representative capability, is becoming better fit to the manifold of well-folding proteins. To allow ourselves to investigate this, we initially chose to train ESM-2 via ESM-3s variable masking ratio, in order to allow it to perform generation by gradual unmasking of the most confident tokens, ala MaskGIT or ESM-3. We then ran a psuedo-perplexity like metric on structure, masking a single token at a time and measuring the average stability of predicted protein. Overall, this saw relatively little difference in . This seems to demonstrate the

most models argmax predictions are on the manifold of decently stable proteins, almost by default – no model was saw a significantly poorer performance on this metric. This is logical: essentially all models avoid blowing up the protein if given the chance.

This raises another question: if our goal was to better calibrate the distribution, and not necessarily supplant the argmax prediction (see how we still consider the true answer to be highest probability in the distribution when constructing the dense one-hot-like label), is it perhaps that our model is better capable of choosing more novel answers without destroying the protein? The answer appears to be yes. If we compare our control with a random model from those fine-tuned with dense labels, in this case Decay=.05, we find that, as we sample from a larger top-k choices from the models prediction, models trained with dense labels appear to generalize outside of the peak of the distribution (argmax, top-1). better. We can see this in Table 4 and 6's higher pLDDT values compared to Table 5 - baseline ESM-2. Their higher k-mer diversity and shannon entropy also suggests they do so while maintaining a higher novelty.

5 Conclusion

Overall, we concluded our experiments showed that increasing the influence of dense labels in the training loss leads to a decline in the model's representative capacity, shown particularly by its ability to predict protein thermostability from embeddings. As shown in the results section, performance deteriorates across MSE, MAE, and R^2 metrics with higher dense-weighting. Notably, this degradation does not appear to be a result of overfitting, as no consistent trend emerges with the increased weight decay in the fine-tuning process. This suggests the model is not simply memorizing the limited dense-labeled data, but rather undergoing a shift in its internal representation priorities.

Despite this reduction in predictive accuracy, models fine-tuned with dense labels exhibit improved alignment with the manifold of well-folding proteins to some degree, especially when assessed through generative capabilities. Using a gradual unmasking approach inspired by MaskGIT, we observed that these models maintain stability even when generating novel sequences—argmax predictions remain structurally sound. More importantly, dense-label fine-tuning appears to enhance the model's ability to explore novel yet plausible protein sequences beyond the top-1 predictions, as demonstrated by higher pLDDT scores, greater k-mer diversity, and increased Shannon entropy of these models compared to the ESM-2 baseline. This suggests a more calibrated and expressive distribution without compromising structural integrity.

Several important limitations within our study must be acknowledged. The dense label dataset used in our study for fine-tuning is relatively small and limited in diversity, restricting the generalizability of our models to broader protein variaties. It may also explain our model's lacking performance in the regression task. While the improvements in novelty and distribution calibration are promising trends, we did not observe strong gains in downstream structure prediction beyond the baseline for top-ranked sequences. This may indicate that the utility of dense labeling in protein representation and generation could be context-dependent. Future work should explore scaling dense labeling to larger and more diverse datasets, as well as integrating structural supervision more directly into the generative training loop.

References

[1] cradle-bio. meltome cluster split [dataset]. https://huggingface.co/datasets/cradle-bio/meltome_cluster_split, 2025. Accessed: Apr. 22, 2025.

[2] Ahmed Elnaggar, Michael Heinzinger, Christian Dallago, Gabriel Rihawi, Yu Wang, Lyudmila Jones, Thomas Gibbs, Tamas Feher, Christoph Angerer, Martin Steinegger, et al. Prottrans: Towards cracking the language of life's code through self-supervised deep learning and high performance computing. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10):7112–7127, 2021.

[3] Noelia Ferruz, Steffen Schmidt, and Birte Höcker. Protgpt2 is a deep unsupervised language model for protein design. *Nature Communications*, 13(1):4348, 2022.

[4] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[5] Zeming Lin, Hanieh Akin, Roshan Rao, Brian Hie, Zhongkai Zhu, William Lu, et al. Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637):1123–1130, 2023.

[6] Yuning Luo, Yixuan Xie, Yuxuan Mao, Yuchen Wang, Chen Fan, William K. Ridgeway, et al. Antigen-specific antibody design and optimization with diffusion-based generative models. https://www.biorxiv.org/content/10.1101/2022.07.10.499510, 2022. bioRxiv preprint.

[7] Ali Madani, Ben Krause, Eric R. Greene, Shamsundar Subramanian, Brian P. Mohr, James M. Holton, et al. Large language models generate functional protein sequences across diverse families. *Nature Biotechnology*, 41(8):1099–1106, 2023.

[8] Joshua Meier, Roshan Rao, Robert Verkuil, Jason Liu, Tom Sercu, and Alexander Rives. Language models enable zero-shot prediction of the effects of mutations on protein function. In *Advances in Neural Information Processing Systems*, volume 34, pages 29287–29303, 2021.

[9] Adam J. Riesselman, John B. Ingraham, and Debora S. Marks. Deep generative models of genetic variation capture the effects of mutations. *Nature Methods*, 15(10):816–822, 2018.

[10] Alexander Rives, Joshua Meier, Tom Sercu, Siddharth Goyal, Zeming Lin, Jason Liu, Demi Guo, Myle Ott, C. Lawrence Zitnick, Jerry Ma, and Rob Fergus. Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15):e2016239118, 2021.

[11] Philip A. Romero, Tracy M. Tran, and Adam R. Abate. Dissecting enzyme function with microfluidic-based deep mutational scanning. *Proceedings of the National Academy of Sciences*, 112(23):7159–7164, 2015.

[12] William P. Russ, Matteo Figliuzzi, Christian Stocker, Pierre Barrat-Charlaix, Mark Socolich, Peter Kast, Donald Hilvert, Rémi Monasson, Simona Cocco, Martin Weigt, and Rama Ranganathan. An evolution-based model for designing chorismate mutase enzymes. *Science*, 369(6502):440–445, 2020.

[13] John L. Watson, David Juergens, Nicholas R. Bennett, Brian L. Trippe, Jason Yim, Harrison E. Eisenach, et al. De novo design of protein structure and function with rfdiffusion. *Nature*, 620(7972):159–170, 2023.